

Css Weighted Least Squares

Component Design Document

1 Description

Coarse sun sensor weighted least squares estimator. Fits a unit sun heading in the body frame to the cosine readings of the coarse sun sensor constellation and differences successive headings into a body rate. Wraps the `CssWeightedLeastSquaresAlgorithm` C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 5
- **Number of Invokee Connectors** - 3
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 4
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 1
- **Number of Data Dependencies** - 1
- **Number of Packets** - *None*

3.2 Diagram

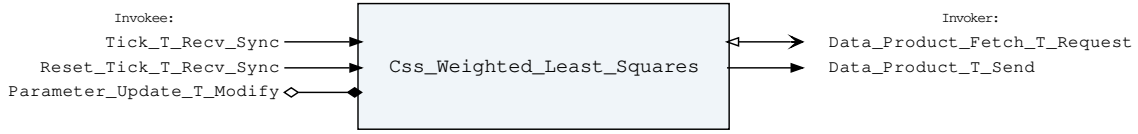


Figure 1: Css Weighted Least Squares component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Css Weighted Least Squares Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Reset_Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Parameter_Update_T_Modify	modify	Parameter_Update.T	-	1

Connector Descriptions:

- **Tick_T_Recv_Sync** - Run the algorithm up to the current time.
- **Reset_Tick_T_Recv_Sync** - Discard the prior heading so no rate is produced until two headings have been observed again. Called on GNC state change so a heading from the previous state is not differenced into a rate.
- **Parameter_Update_T_Modify** - The parameter update connector.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Css Weighted Least Squares Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_Fetch_T_Request	request	Data_Product_Fetch.T	Data_Product_Return.T	1
Data_Product_T_Send	send	Data_Product.T	-	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Csw Weighted Least Squares Set Id Bases Parameters

Name	Type
<code>Data_Product_Id_Base</code>	<code>Data_Product_Types.Data_Product_Id_Base</code>
<code>Parameter_Id_Base</code>	<code>Parameter_Types.Parameter_Id_Base</code>

Parameter Descriptions:

- **Data_Product_Id_Base** - The value at which the component's data product identifiers begin.
- **Parameter_Id_Base** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Csw Weighted Least Squares Map Data Dependencies Parameters

Name	Type
<code>Css_Sensor_Input_Id</code>	<code>Data_Product_Types.Data_Product_Id</code>
<code>Css_Sensor_Input_Stale_Limit</code>	<code>Ada.Real_Time.Time_Span</code>

Parameter Descriptions:

- **Css_Sensor_Input_Id** - The external data product ID used to fetch the value for the 'Css_Sensor_Input' data dependency.
- **Css_Sensor_Input_Stale_Limit** - The stale limit used for the 'Css_Sensor_Input' data dependency. If the fetched data product is older than the given `Time_Span` then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the estimator with the control period and the default parameter values. The `init` subprogram requires the following parameters:

Table 5: Csx Weighted Least Squares Implementation Initialization Parameters

Name	Type	Default Value
Control_Period	Basic_Types.Positive_Short_Float	<i>None provided</i>

Parameter Descriptions:

- **Control_Period** - [s] Time between two algorithm updates. The body rate is the change in heading over this much time, so it must equal the rate at which the component is scheduled. Fixed by the assembly, so it is not a runtime parameter. The type keeps it greater than zero.

3.6 Commands

The Csx Weighted Least Squares component has no commands.

3.7 Parameters

Parameters for the Csx Weighted Least Squares component.

Table 6: Csx Weighted Least Squares Parameters

Local ID	Parameter Name	Type	Default Value
0x0000 (0)	Csx_N_Hat_B	Packed_F32x24.T	[+0.70710678, -0.50000000, +0.50000000, +0.70710678, -0.50000000, -0.50000000, +0.70710678, +0.50000000, -0.50000000, +0.70710678, +0.50000000, -0.50000000, +0.70710678, +0.50000000, -0.70710678, +0.00000000, +0.70710678, -0.70710678, +0.70710678, +0.00000000, -0.70710678, +0.00000000, -0.70710678, -0.70710678, -0.70710678, +0.00000000]

0x0001 (1)	Css_Availability	Css_Availability_X8.T	[others=>Css_Weighted_Least_Squares_Enums.Sensor_Availability.Available]
0x0002 (2)	Use_Measurements_As_Weights	Packed_Boolean.T	(Value=>False)
0x0003 (3)	Sensor_Use_Thresh	Packed_F32.T	(Value=>0.15)

Parameter Descriptions:

- **Css_N_Hat_B** - [-] Boresight unit vector of each coarse sun sensor in body frame components, three per sensor in row major order. An available sensor's boresight must be a unit vector to within 1e-3.
- **Css_Availability** - [-] Which coarse sun sensors take part in the fit. An unavailable sensor is left out and its boresight is never read.
- **Use_Measurements_As_Weights** - [-] Whether each sensor's reading becomes its own weight in the least squares fit.
- **Sensor_Use_Thresh** - [-] Cosine threshold at or below which a reading is dropped from the fit. Must lie in [0, 1].

3.8 Events

The Css Weighted Least Squares component has no events.

3.9 Data Products

Data products for the Css Weighted Least Squares component.

Table 7: Css Weighted Least Squares Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Sun_Heading_Estimate	Css_Weighted_Least_Squares_Output.T

Data Product Descriptions:

- **Sun_Heading_Estimate** - [-] The estimated sun heading and body rate, the post-fit residuals, and the count of sensors that contributed to the fit.

3.10 Data Dependencies

Data dependencies for the Css Weighted Least Squares component.

Table 8: Css Weighted Least Squares Data Dependencies

Data Dependency Name	Type
Css_Sensor_Input	Css_Sensor_Values.T

Data Dependency Descriptions:

- **Css_Sensor_Input** - [-] Corrected cosine reading of each coarse sun sensor slot, produced by the css_comm component.

3.11 Packets

The Css Weighted Least Squares component has no packets.

3.12 Faults

The Css Weighted Least Squares component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Css_Weighted_Least_Squares_Tests* Test Suite

This is a unit test suite for the Css Weighted Least Squares component

Test Descriptions:

- **Test** - Fit the sun heading along each body axis from the Python reference test, with and without measurement weights, to ensure the Ada to C to C++ integration is sound.
- **Test_Partial_Coverage** - Check the minimum norm fits with two and one lit sensors, the effect of an unavailable sensor, the residual indexing, and the no signal case against the Python reference model.
- **Test_Rate_Estimate** - Check the body rate from two successive headings, including a slow slew, a heading reversal, and the reset connector.
- **Test_Reconfigure** - Check that a parameter update changes the live fit, and that the fit weights do not carry over from a cycle with more lit sensors.
- **Test_Invalid_Parameter** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **Test_Invalid_Data_Dependency** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Css_Availability_X8.T:

Availability of each of the eight coarse sun sensor slots, one byte per slot as the C boundary carries it. Values outside the enumeration fail type validation during parameter staging.

Table 9: Css_Availability_X8 Packed Array : 64 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Css_Weighted_Least_Squares_Enums.Sensor_Availability.E	0 => Available 1 => Unavailable	8	8	64

Css_Sensor_Values.T:

Corrected CSS sensor cosine values for the 8 physical CSS ADC channels.

Table 10: Css_Sensor_Values Packed Record : 512 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Data	Packed_F64x8.T	-	512	0	511

Field Descriptions:

- **Data** - *No description provided.*

Css_Weighted_Least_Squares_Output.T:

One cycle of the coarse sun sensor weighted least squares estimate. Maps to `CssWeightedLeastSquaresOutput_c` in the `cssWeightedLeastSquares` C shim, which returns it by value.

Table 11: Css_Weighted_Least_Squares_Output Packed Record : 480 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sun_Heading_B	Packed_F32x3.T	-	96	0	95
Omega_Bn_B	Packed_F32x3.T	-	96	96	191
Post_Fit_Residuals	Packed_F32x8.T	-	256	192	447
Num_Css_Viewing_Sun	Interfaces.Unsigned_32	0 to 4294967295	32	448	479

Field Descriptions:

- **Sun_Heading_B** - [-] Estimated unit sun heading in body frame components. Zero when no fit was possible.
- **Omega_Bn_B** - [rad/s] Inertial angular velocity in body frame components, from two successive headings. Rotation about the sun heading is not observable. Zero until two headings have been observed.
- **Post_Fit_Residuals** - [-] Post-fit residuals, one per sensor that contributed, packed into the leading `Num_Css_Viewing_Sun` entries. The rest are zero.
- **Num_Css_Viewing_Sun** - [-] Sensors that contributed to the fit this cycle: available, and reading above the use threshold.

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 12: Data_Product Packed Record : 1096 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_Header.T	-	72	0	71	-
Buffer	Data_Product_Types.Data_Product_Buffer_Type	-	1024	72	1095	Header.Buffer_Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 13: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types.Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 14: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types.Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_Types.Data_Product_Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 15: Data_Product_Return Packed Record : 1104 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_ Status	Data_ Product_ Enums. Fetch_ Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_ Data_ Product	Data_ Product. T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Packed_Boolean.T:

Packed record holding a boolean type.

Table 16: Packed_Boolean Packed Record : 8 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Boolean	0 => False 1 => True	8	0	7

Field Descriptions:

- **Value** - The boolean value.

Packed_F32.T:

Single component record for holding packed 32-bit floating point number.

Table 17: Packed_F32 Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Short_Float	-3.40282e+38 to 3.40282e+38	32	0	31

Field Descriptions:

- **Value** - The 32-bit floating point number.

Packed_F32x24.T:

Packed 24 element vector of 32-bit floats. Holds a table of 8 three-component vectors in row major order.

Table 18: Packed_F32x24 Packed Array : 768 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	24	768

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 19: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Packed_F32x8.T:

Packed 8 element vector of 32-bit floats. Used for thruster command arrays sized to the system thruster count.

Table 20: Packed_F32x8 Packed Array : 256 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	8	256

Packed_F64x8.T:

Packed 8 element vector of 64-bit floats.

Table 21: Packed_F64x8 Packed Array : 512 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Long_Float	-inf to inf	64	8	512

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 22: Parameter Packed Record : 792 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_Header.T	-	24	0	23	-
Buffer	Parameter_Types. Parameter_Buffer_Type	-	768	24	791	Header.Buffer_Length

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 23: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15
Buffer_Length	Parameter_Types. Parameter_Buffer_ Length_Type	0 to 96	8	16	23

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 24: Parameter_Update Packed Record : 824 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Table_Id	Parameter_Types. Parameter_Table_Id	0 to 65535	16	0	15	–
Operation	Parameter_Enums. Parameter_Operation_Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate	8	16	23	–
Status	Parameter_Enums. Parameter_Update_Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	–
Param	Parameter.T	–	792	32	823	–

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 25: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of $1/(2^{16})$ sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 26: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Css_Weighted_Least_Squares_Enums.Sensor_Availability.E:

Availability of a single coarse sun sensor. Values match the C DeviceAvailability_c enumeration, so Available is zero rather than the more obvious one.

Table 27: Sensor_Availability Literals:

Name	Value	Description
Available	0	The sensor takes part in the fit when its reading is above the use threshold.
Unavailable	1	The sensor is excluded from the fit and its boresight is never read.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 28: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 29: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.
Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 30: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.